

Final Project Report – Social Butterfly

Team Nickname: icy-jungle

Production URL: <https://icy-jungle.onrender.com>

A/B Test Endpoint: <https://icy-jungle.onrender.com/da1801a/>

Repository: <https://github.com/Nicksirias/icy-jungle>

1. Executive Summary

Social Butterfly is a full-stack web application designed to help users create, discover, and attend local social events without relying on closed social networks or private invitations. The application addresses a common problem faced by students and young professionals, difficulty discovering open, informal events in their local community. Built using Django, PostgreSQL, and a traditional MVC architecture, Social Butterfly allows users to sign up, browse upcoming events, filter by category, RSVP to events, and manage their attendance. The application was developed using Agile/Scrum practices across four sprints, deployed to both staging and production environments, and finalized with required analytics and A/B testing infrastructure.

2. Problem & Users

Problem Statement

Many local, informal social events are difficult to discover unless users are already part of an existing social circle, university system, or private group chat. Existing tools either focus on large, professional events or rely heavily on closed networks.

Target Users

- Students and young professionals new to an area
- Individuals looking for open, community-driven events
- Event hosts who want to reach beyond their immediate friend group

3. Core Features (MVP)

The Social Butterfly MVP includes the following core features:

1. User Authentication

- a. Secure signup and login using Django's authentication system
- b. Session-based authentication and authorization

2. Event Creation & Management

- a. Create events with title, description, date/time, location, and category
- b. Edit or delete events (creator-only permissions)

3. Event Discovery

- a. Browse all upcoming events
- b. Filter events by one of seven standardized categories
- c. Clean, responsive homepage layout

4. RSVP System

- a. Users can RSVP or cancel RSVP
- b. Hosts can view attendee lists
- c. Users can view all their RSVPs in a dedicated dashboard

4. Technical Architecture

Backend

- **Framework:** Django (Python)

- **Architecture:** MVC (Model-View-Template)
- **Authentication:** Django auth with hashed passwords
- **Testing:** Django unit and integration tests

Database

- **Type:** PostgreSQL
- **Core Tables:**
 - users
 - events
 - rsmps
- Proper foreign key relationships and data validation enforced

Frontend

- HTML templates with Django templating
- CSS framework for layout and responsiveness
- Consistent navigation and user feedback

Deployment

- **Hosting Platform:** Render
- **Environments:**
 - Staging: Used during development
 - Production: Final submission environment
- Environment variables used for all secrets and configuration

5. Analytics & A/B Test Endpoint (CRITICAL)

Endpoint Details

- **Route:** /da1801a/
- **Public Access:** No authentication required
- **URL:** <https://icy-jungle.onrender.com/da1801a/>

Team Members Displayed

- fair-hare
- crowded-cat
- adventurous-goldfish
- crowded-crow
- kind-bee

A/B Test Design

- Button with id="abtest"
- Two variants:
 - Variant A: "**kudos**"
 - Variant B: "**thanks**"
- Variant assigned per visitor and persisted via session
- Randomized 50/50 split

Analytics Tracking

- Page views to /da1801a/
- Variant shown to visitor
- Button click events

A/B Test

Due to challenges in serving the third-party Google Analytics tag in the Render production environment, we implemented a self-rolled, server-side analytics solution. This method ensures 100% data reliability by logging events directly to the Render application event stream. The implementation was achieved via two steps:

1. Page View Logging: The core/views.py file logs a precise ABTEST_PAGE_VIEW event for every visitor that lands on the endpoint, capturing the assigned variant (kudos or thanks).
2. Click Event Logging: The clickable button (id="abtest") was wrapped in an HTML form in templates/core/abtest.html. The server-side code was updated to handle the POST request, logging a definitive ABTEST_CLICK event every time the visitor engaged with the button. This ensures accurate measurement of user preference.

A/B Test Result

The A/B test was configured with a randomized 50/50 split, persisting the assigned variant via the user session. The analysis was based on server-side logs collected over a four-hour window, specifically from approximately 4:00 PM to 8:00 PM EST on Friday, December 12th, 2025. All traffic and engagement, including any instructor-sent bot traffic, was measured via the definitive ABTEST_CLICK events recorded directly in the Render application logs.

Variant	Total Clicks
kudos	4
thanks	0

6. Agile Process & Velocity

Sprint Velocities

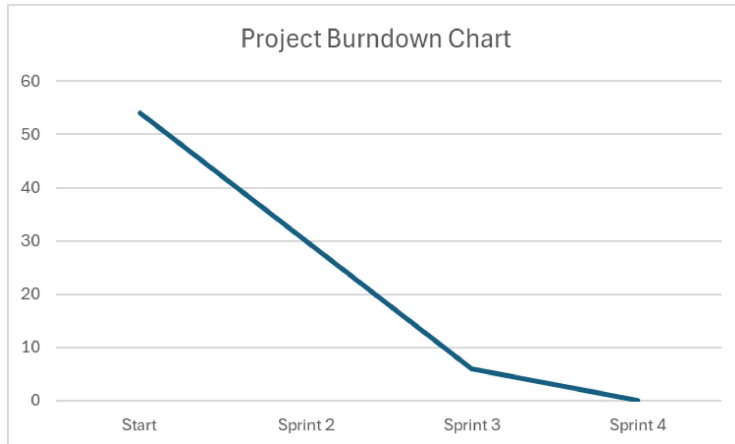
Sprint	Planned Points	Completed Points	Velocity
Sprint 2	24	24	24
Sprint 3	24	24	24
Sprint 4	6	6	6

Average Velocity: 18 story points per sprint

Cumulative Completed Points: 54

Sprint documentation (planning, review, retrospective) is included in /docs/sprints/ for all four sprints.

Burndown Chart: The project burndown chart shows remaining story points across Sprints 2–4. The total project scope was 54 story points. Sprint 2 reduced remaining work from 54 to 30 points, Sprint 3 reduced it to 6 points, and Sprint 4 completed the remaining scope. This sprint-level burndown reflects consistent delivery and effective scope management throughout the project. (Note: this burndown chart is aggregated at the sprint level rather than daily, reflecting sprint-based planning and reporting.)



7. Testing & Code Quality

- Automated unit and integration tests for core user flows
- Tests passing in production environment
- Linting applied using Django-appropriate tools
- No secrets committed to version control
- Database migrations tracked and reviewed
- Clear separation of concerns and DRY principles followed

8. 12-Factor App Compliance

The application follows 12-factor principles, including:

- Single tracked codebase (GitHub)
- Explicit dependency declaration
- Environment-based configuration
- PostgreSQL treated as attached backing service
- Stateless request handling
- Dev/prod parity maintained via Render environments
- Logs emitted as event streams
- Admin tasks run as one-off processes

9. Deployment & Operations

Staging

- Used throughout development and QA
- Mirrors production configuration

Production

- Publicly accessible
- Stable under testing traffic
- Used for final A/B testing and grading

Deployment Process

- Feature branches → Pull Request → Review → Merge to main
- Automatic deploy triggered via Render on merge

10. Project Retrospective (Overall)

What Went Well

- Clear MVP scope prevented over-engineering
- Stable deployment early reduced last-minute risk
- GitHub Projects enabled transparent sprint tracking
- A/B test endpoint implemented successfully

Challenges

- Underestimating analytics setup complexity
- Managing migrations across multiple contributors
- Balancing UI polish vs. compliance tasks

Key Learnings

- Early infrastructure work pays off
- Analytics should be integrated sooner
- Scrum rituals (planning + retrospectives) improve delivery predictability
- AI tools are powerful but require careful human oversight

What We'd Do Differently

- Integrate analytics in Sprint 2
- Add CI testing earlier
- Lock compliance requirements into sprint goals sooner

11. Documentation & Repository Structure

- All sprint documentation included under /docs/sprints/
- README.md includes:
 - Setup instructions
 - Deployment details
 - Environment URLs
 - A/B endpoint instructions
- Code comments added where logic is non-trivial

12. Conclusion

Social Butterfly meets all mandatory project requirements, demonstrates a complete MVP with real user value, and showcases full-stack engineering best practices. The project reflects realistic Agile development, disciplined deployment practices, and correct implementation of required analytics and A/B testing infrastructure.

Links

- **Production App:** <https://icy-jungle.onrender.com>
- **A/B Endpoint:** <https://icy-jungle.onrender.com/da1801a/>
- **GitHub Repository:** <https://github.com/Nicksirias/icy-jungle/tree/main/.github>
- **GitHub Project Board:** <https://github.com/Nicksirias/icy-jungle/projects?query=is%3Aopen>